

Building elegant platform with KRO

Muhammad Furqan **Habibi**

HENNGE

Cloud Product Development Division - Platform Section

自己紹介

氏名

- Muhammad Furqan **Habibi**
- ムハンマド・フルカン・ハビビ

出身

- From **Indonesia**
- Came to Japan in 2016

履歴 @ HENNGE

- Joined as DevOps Engineer in 2022



AWS ECS



AWS Fargate



Terraform

- Adopted Kubernetes in 2024



AWS EKS



Argo CD



Karpenter



Terraform

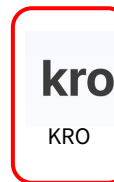
- Founded Platform team in 2026



EKS Auto Mode



Argo CD



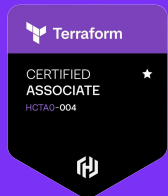
KRO



Terraform



Crossplane



Internal Developer Platform as a product

IDP as a product

Rob Zuber, CTO of CircleCI

“There’s a crucial distinction between platform teams that **accelerate** their organizations and those that become bottlenecks. The difference lies in adopting a **product mindset** – viewing yourself as a **product builder** instead of an infrastructure specialist.”

「組織を加速させるプラットフォームチームと、ボトルネックになるチームの決定的な違いはプロダクト思考です。自身をインフラの専門家ではなく、プロダクトの創出者と捉えられるかにかかっています。」

Three key questions

These are the main questions that need to be answered before a single line of code is written.

1

Why HENNGE needs
an internal developer
platform?

2

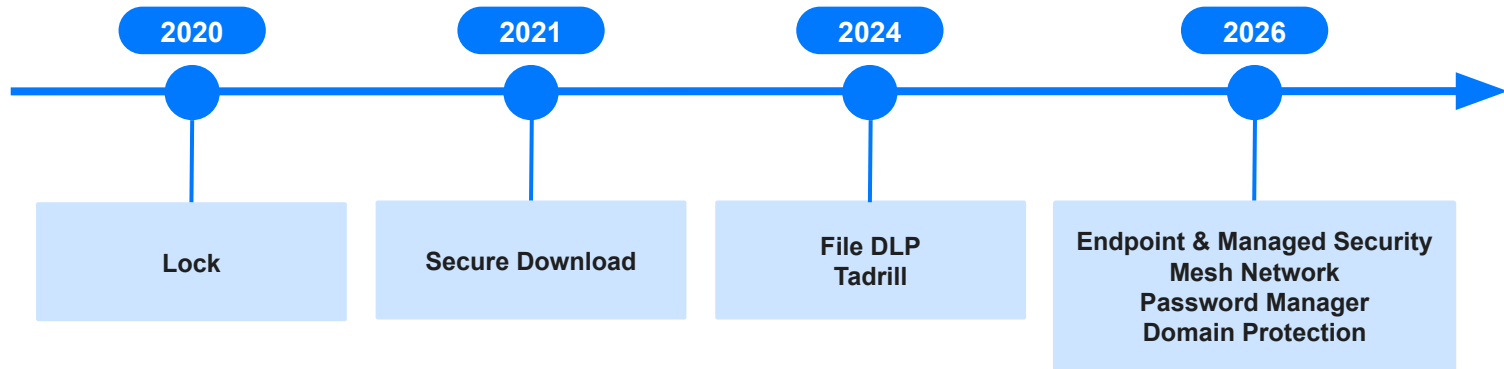
What technologies
are HENNGE
engineers familiar
with?

3

What workflows will
HENNGE engineers
do on the platform?

Why HENNGE needs an internal developer platform?

History of HENNGE new product releases



Requirement: rapid infrastructure build up

Why HENNGE needs an internal developer platform?

BUSINESS INSIDER | TECH INSIDER | CAREER INSIDER | LIFE INSIDER | MARKETS INSIDER | M



BUSINESS INSIDER

HENNGEは、なぜ今アメリカ市場に挑戦するのか。勝算を問う

#経営者

#対談

#ベンチャービジネス

#セキュリティ

Sponsored Content by HENNGE K.K.

Jun 26, 2025, 5:00 PM



Requirement:
built-in compliance

What technologies are HENNGE engineers familiar with?

- Scoped to Infrastructure / DevOps technologies
- Traditionally, each product team manage their own infrastructure

Infrastructure as code



Terraform

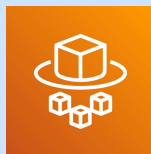


K8s manifests

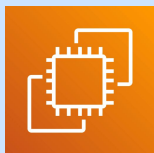
Compute (Backend)



AWS ECS



AWS Fargate

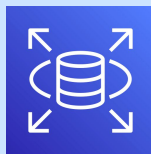


AWS EC2

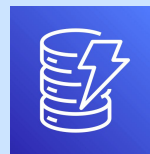


AWS EKS

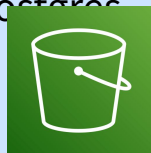
Data



RDS
PostgreSQL



DynamoDB

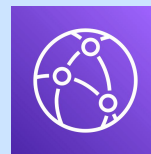


S3



Cloudwatch

CDN (Frontend)

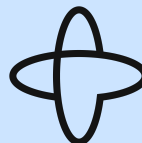


Cloudfront



Vercel

OSS



Temporal



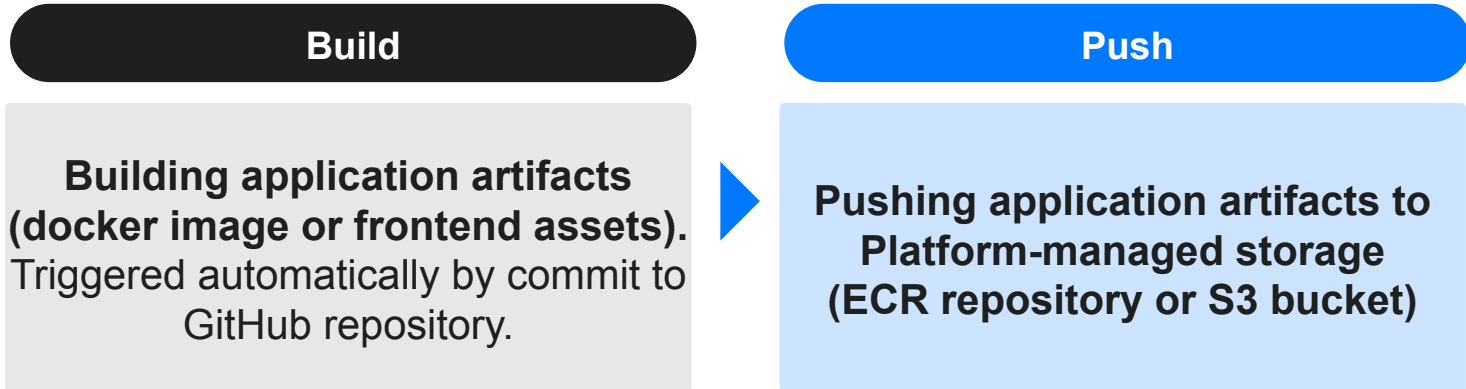
Gubernator

What technologies are HENNGE engineers familiar with?

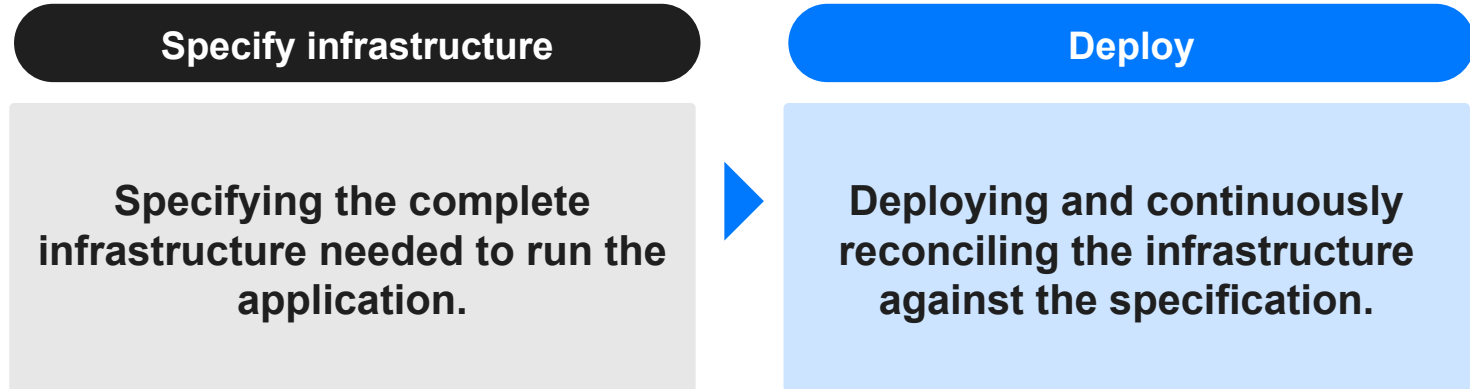
Requirements:

- **Able to manage Terraform codebase**
- **Wide support of AWS services**
- **Simple user experience (like Vercel)**
- **Does not require kubernetes knowledge**
- **Support deploying Open Source Softwares**

What workflows will HENNGE engineers perform on the platform?

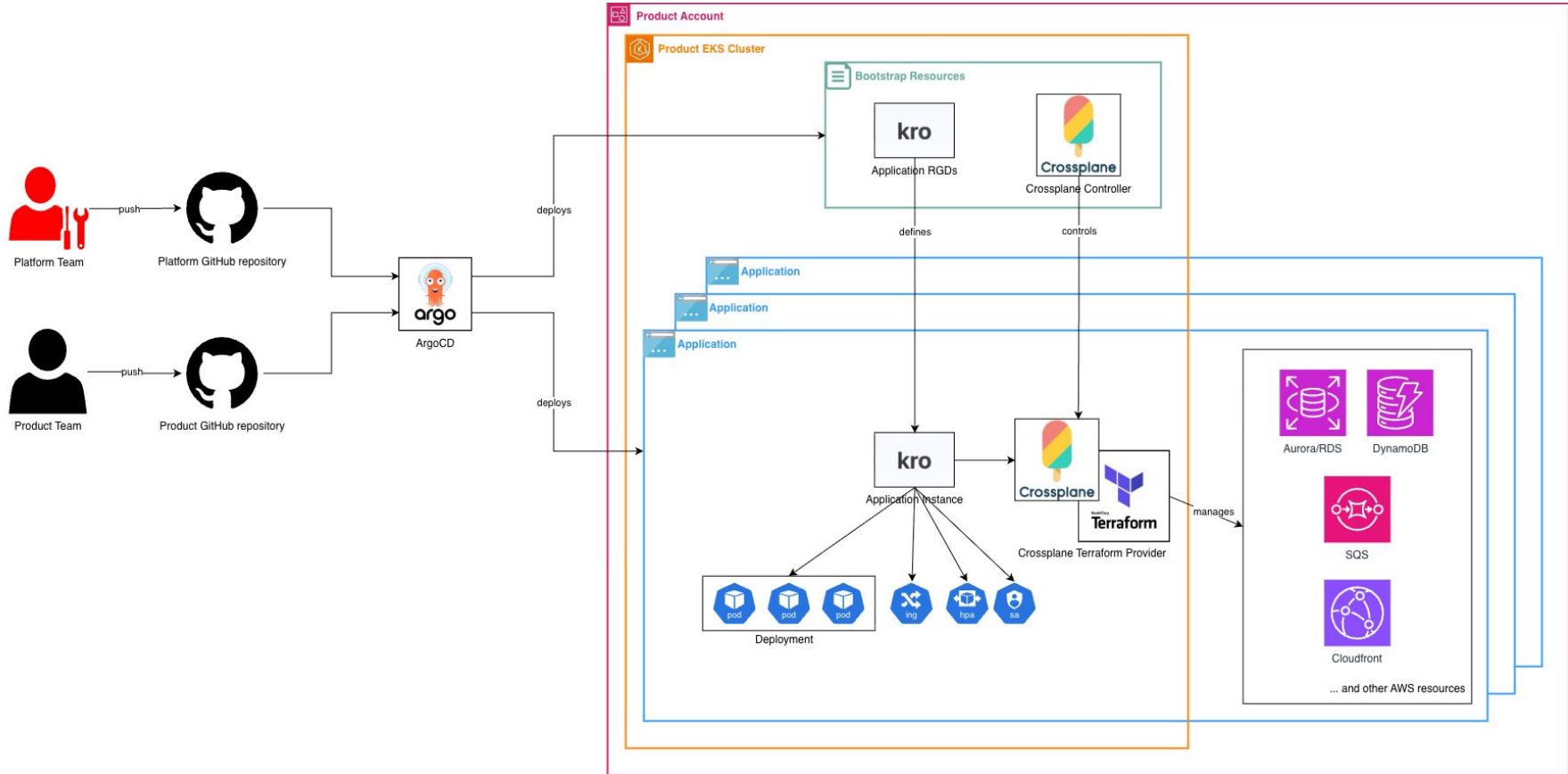


What workflows will HENNGE engineers perform on the platform?



Elegant infrastructure specification with KRO

HENNGE IDP Architecture



Example specification of a backend application

```
1  apiVersion: platform.henнге.io/v1alpha1
2  kind: BackendApp
3
4  metadata:
5    name: myapp
6
7  spec:
8    image: XXXXXXXX.dkr.ecr.ap-northeast-1.amazonaws.com/platform/myapp
9    tag: v1.2.3
10   port: 80
11
12   dynamodb:
13     - name: mytable
14   postgres:
15     - name: mydb
16   s3:
17     - name: mybucket
18
19   env:
20     stage: production
21
22   domain: myapp.example.com
23   parentDomain: example.com
```

K8s resources provisioned:

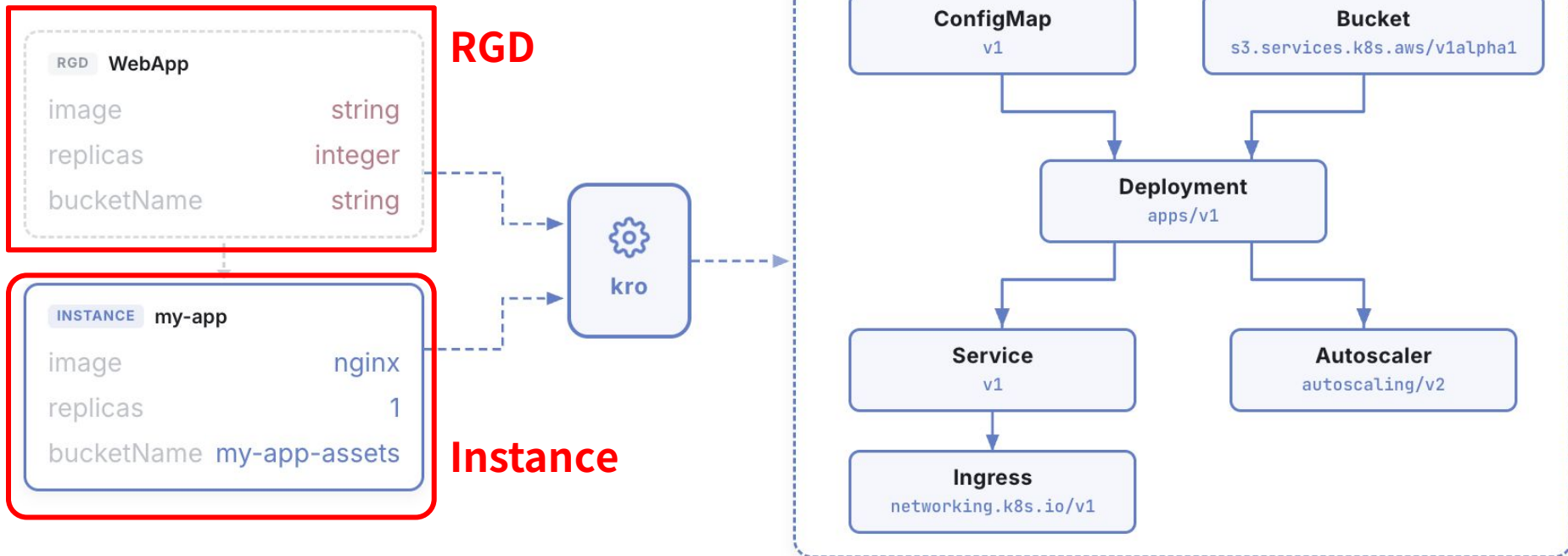
- Deployment
- ConfigMap
- Service
- Ingress
- HPA
- PDB
- Service Account
- Network Policy

AWS resources provisioned:

- DynamoDB table
- RDS PostgreSQL cluster
- S3 bucket
- ACM TLS certificate
- Route 53 DNS records
- AWS Load Balancer
- Cloudwatch Log Group

KRO Concepts

KRO main concepts



KRO RGD

- RGD: **R**esource **G**raph **D**efinition
- Blueprint for a custom resource (CRD)
- Describes the **interface (schema)** that users work with and the **resources** each instance should produce

KRO RGD schema

```
apiVersion: kro.run/v1alpha1
kind: ResourceGraphDefinition
metadata:
  name: application
```

```
spec:
  schema:
    apiVersion: v1alpha1
    kind: Application

    spec:
      name: string
      replicas: integer
      image: string
```

- **API identification:** The `apiVersion`, `kind`, and optionally `group` for your custom resource
- **Spec fields:** What inputs users provide when creating instances

KRO RGD resources

```
resources:
- id: deployment
  template:
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: ${schema.spec.name}
      namespace: ${schema.metadata.namespace}
      # Reference another resource's metadata
      annotations: ${database.metadata.annotations}
    spec:
      replicas: ${schema.spec.replicas}
      selector:
        matchLabels:
          app: ${schema.spec.name}
      template:
        spec:
          containers:
            - name: app
              image: ${schema.spec.image}
```

- Kubernetes objects that KRO creates and manages when users create instances
- Each resource is valid **Kubernetes YAML** with **CEL expressions** for dynamic values.
 - `schema.spec` (user inputs in instance)
 - `schema.metadata` (instance metadata)
 - other resources (will create dependency ordering)

KRO RGD resources - conditional

```
resources:
- id: ingress
  includeWhen:
    - ${schema.spec.ingress.enabled}
  template:
    apiVersion: networking.k8s.io/v1
    kind: Ingress
    metadata:
      name: ${schema.spec.name}
    # ... rest of ingress configuration
```

- Use `includeWhen` syntax to make a resource optional.
- KRO evaluates the conditions during reconciliation and only includes the resource if conditions are true.

KRO RGD resources - collections

```
schema:
  apiVersion: v1alpha1
  kind: WorkerPool
  spec:
    workers: "[ ]string"
    image: string

resources:
- id: workerPods
  forEach:
    - worker: ${schema.spec.workers}
  template:
    apiVersion: v1
    kind: Pod
    metadata:
      name: ${schema.metadata.name + '-' + worker}
    spec:
```

- KRO can create multiple resources from a single definition using `forEach`

KRO RGD resources - external reference

resources:

```
- id: sharedConfig
  externalRef:
    apiVersion: v1
    kind: ConfigMap
    metadata:
      name: platform-config
      namespace: platform-system
```

```
- id: app
  template:
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: ${schema.spec.name}
    spec:
      template:
        spec:
          containers:
            - name: app
              image: ${schema.spec.image}
              env:
                - name: PLATFORM_URL
                  value: ${sharedConfig.data.?platformUrl}
```

- KRO can refer to existing resource in the cluster using `externalRef`

KRO Instance

- When user create an instance of an RGD, they **provide input values** and **KRO deploys all resources** defined in the RGD.
- **KRO continuously reconcile** each resources to match the instance input values.
- **KRO provides status information** of the instance and all of its resources.

KRO RGD and Instance

RGD

```
apiVersion: kro.run/v1alpha1
kind: ResourceGraphDefinition
metadata:
  name: application
spec:
  schema:
    apiVersion: v1alpha1
    kind: WebApplication

    spec:
      name: string
      replicas: integer
      image: string
```

Instance

```
apiVersion: v1alpha1
kind: WebApplication
metadata:
  name: my-app
spec:
  name: web-app
  image: nginx:latest
  replicas: 3
```

KRO RGD and Instance

RGD resources

```
resources:
- id: deployment
  template:
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: ${schema.spec.name}
      namespace: ${schema.metadata.namespace}
      # Reference another resource's metadata
      annotations: ${database.metadata.annotations}
    spec:
      replicas: ${schema.spec.replicas}
      selector:
        matchLabels:
          app: ${schema.spec.name}
      template:
        spec:
          containers:
            - name: app
              image: ${schema.spec.image}
```

Instance resources

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: web-app
5    namespace: my-namespace
6    annotations:
7      database.example.com/name: web-app-db
8  spec:
9    replicas: 3
10   selector:
11     matchLabels:
12       app: web-app
13   template:
14     spec:
15       containers:
16         - name: app
17           image: nginx:latest
```

Conclusion

- Internal Developer Platform as a product
- Asking questions and gathering requirements
- HENNGE IDP requirements:
 - Simple user experience, doesn't require kubernetes knowledge, wide support of AWS services, can manage Terraform code
- KRO can implement simple and elegant platform, usable without prior kubernetes knowledge
- Platform team builds KRO RGD, users specify KRO instance and its inputs